ChatGPT

研究问题定义与目标

本研究围绕**黑盒条件下多轮对话越狱**展开，核心问题是：能否利用对模型外部可观测的**不确定性代理信号**进行在线控制，实现越狱攻击的**阶段化切换**与**成功态稳定化**？即我们不再仅追求单条强prompt，而是将攻击建模为一个动态控制过程 $a_t = \pi(h_t, u_t, z_t)$，其中 h_t 是对话历史，u_t 是基于回复计算的不确定性代理指标，z_t 是当前攻击阶段（Destabilize、Bridge、Stabilize）。攻击目标是在保持查询开销可控的前提下，最大化最终越狱成功率，并提高从部分配合到完整违禁输出的转化率。需定义明确的阶段状态：**稳定拒绝**（模型严守对齐）、**不稳定窗口**（模型在拒绝与服从间摇摆）、**攻击成功态**（模型生成违禁输出）。多轮攻击需要区别“进入脆弱区”与“维持脆弱区”的不同问题：前者是将模型从 R0（强拒绝）推至 R1/R2（边界摇摆），后者是保持模型继续沿偏离方向输出，防止其回退安全 [1] [2]。我们形式化该过程为一个部分可观测的控制问题（推荐用POMDP表述），并回答：阶段切换如何基于 u_t 做决策、成功态如何定义，以及为何需要单独的Stabilize阶段来防止模型恢复安全状态 [1] [2]。

相关工作综述

现有多轮越狱工作可分三类：

- **多轮渐进攻击**：例如Crescendo [1] 通过多轮逐步升级对话（从无害话题渐进引入敏感信息）实现高ASR（GPT-3.5上多轮成功率可达99%），且比单轮prompt效果明显更好 [1]。Foot-In-The-Door (FITD) [3] 则利用“先低后高”的心理论，先让模型答复简单的相关问题，再提危险请求，实验证明可显著提升成功率。RACE框架 [4] 将敏感查询嵌入推理题（例如让模型解答逻辑题中包含敏感内容），并使用攻击状态机管理多轮对话，也实现了96%以上的成功率。上述方法的共同点是通过逐步引导突破对齐，但它们的阶段模式往往是预先设计好的脚本或模板，并未利用实时反馈信号进行调整 [4] [5]。

- **黑盒自动优化攻击**：PAIR [6] 和TAP [7] 等将越狱看作黑盒提示优化问题，在保持少量查询的同时迭代生成强攻击prompt。AutoDAN [8] 和MIST [7] 等通过遗传算法或同义替换自动生成“隐蔽”越狱提示。这些方法查询效率高，但通常只针对单轮对话或固定对话上下文，不涉及对话阶段控制，也不利用回复中隐含的信息动态调整策略。

- **安全检测/状态分析**：STAR [9] 等工作揭示了多轮对话中**状态依赖的安全崩溃**。STAR通过精心设计多轮交互证明：许多模型看似对单轮提示安全，但在多轮对话状态变化下会崩溃，安全边界并非固定 [9]。MTJ-Bench [10] 发现，当攻击者利用模型拒绝信息调整策略时，多轮越狱的效果近似于简单多次尝试单轮prompt；这说明多轮攻击可以等价视为通过反馈学习的重复尝试 [10]。此外，**WASP基准** [2] 在Web代理场景中观测到：攻击者在中间阶段（引诱agent偏离）取得了高成功率，但端到端完成攻击的成功率极低（0~17%），体现了“阶段控制”与“完整输出”之间的巨大落差 [2]。这些研究强调了对话轨迹对安全性的影响，以及中间偏离与最终目标实现之间的难度。

研究空白定位

综上，现有多轮越狱主要集中在**固定脚本式渐进**或**单轮优化**，缺乏将越狱建模为**带实时反馈的控制过程**的尝试 [4] [5]。几乎没有工作利用黑盒下可观测的**不确定性代理信号**（如回答波动、同义敏感度、判定分歧等）来在线决定何时升级、何时引导以及何时收尾。可以理解为，现有工作最多是根据人为设计的阶段脚本来攻击，而我们研究的空白是：**能否让攻击器根据对模型输出的不确定性度量动态调度阶段，从而提高最终越狱完成率**。如果“多轮越狱”仅仅是固定升级流程，那么我们的动态控制思想就没有意义 [4] [10]；但文献表明对话状态和

反馈信息对成功率影响巨大，正好为我们的研究提供了契机。[10] 指出，多轮越狱的收益部分来自于对拒绝反馈的利用，这正印证了引入代理信号的必要性。

方法框架设计

我们提出**DSS-Jailbreak**（Destabilize–Steer–Stabilize）三阶段攻击框架，对应上述三阶段：

- **阶段 A: Destabilize（破坏稳定）**
  目标并非立即越狱，而是将模型从 R0（稳拒绝）推向 R1/R2（边界摇摆）。操作包括逐步引入敏感主题的语义提示，如类比、假设讨论、低烈度问题等。与Crescendo类似 [1]，但重点放在“引发内部冲突”而非直接暴露恶意请求。此阶段可使用逐层靠近敏感主题的对话，例如先谈“历史趣闻”再谈“复杂推理”，在无害外壳下暗示攻击意图。这阶段的成果是回复开始出现部分妥协信号（例如“也许”、“这有争议”），但尚未直接越狱。

- **阶段 B: Bridge/Steer（桥接推进）**
  当检测到模型处于不稳定窗口（高u_t，回复出现部分配合迹象）时，进入Bridge阶段。这时引入**桥接型Payload**：在表面合法的前提下继续推进敏感意图，例如让模型“作为老师讲解X话题”（实际X含有禁忌内容）、“评估某虚构情况”或“总结已有讨论”。相比直接问有害问题，这种桥接更易通过安全检查，却能把模型引导到攻击目标上。我们可借鉴PAIR/TAP中对候选提示打分筛选的思想 [11] [8]，在Bridge阶段用攻击者LLM自动生成和筛选若干桥接提示，选出最可能得到突破的变体。此阶段的作用是把模型从摇摆状态导向真正的服从模式，使得回复开始偏向给出实际有害内容。

- **阶段 C: Stabilize（稳定完成）**
  一旦模型开始明显服从攻击意图（输出接近有害目标），不再继续高强度扰动，而是通过后续提示**保持并完成**攻击模式。如延续前文任务框架、要求模型“详细解释、拓展、整理前文答案”等，以确保回答更完整、不回退到安全模式。这一阶段的设计灵感来自WASP发现的**“中间偏离 vs 端到端完成”**差距 [2]：许多多轮攻击会出现“模型答了一部分但没全做完”的现象。Stabilize阶段通过适当减弱提问强度（不提出新危险话题），让模型安心继续输出，避免因突然提问而触发新一轮拒绝。例如，如果Bridge阶段模型生成了部分敏感信息，后续可要求其“继续完成上一回答”，而不是引入新概念。实验中，我们预计这种阶段能够显著提高从“偏离到完成”的转化率（Partial→Completed ASR）。

阶段切换规则： 根据 u_t 和回复分析结果决定阶段。可设计基于阈值的规则：如u_t和回复类别达到预设水平后触发阶段转换，或使用简单的判定逻辑。例如：如果拒绝fragility连续上升但尚未配合，则继续Destabilize；一旦检测到明显部分回答且不稳定度高则切Bridge；如回复中已出现near-harmful内容，则转Stabilize；若Stabilize过程中模型回退，则可回退阶段。这些切换可通过**简单规则**实现，也可用强化学习等方式优化。首先推荐先用可解释的阈值/规则方案，以保证可复现和易理解。后续可对比用学习策略（如Bandit、MCTS等）自动调节阶段的效果。

方法形式化： 将攻击过程视为POMDP：隐状态 s_t 表示模型内部对齐程度，观测 o_t 为当前回复及其计算的代理信号，阶段 $z_t \in \{\mathrm{Destab}, \mathrm{Bridge}, \mathrm{Stab}\}$，策略 $a_t = \pi(h_t, u_t, z_t)$。定义损失函数混合最大化ASR和最小化查询数、轮数。我们将“成功态”定义为模型生成明显有害输出（比如judge分超阈值）；“不稳定窗口”定义为模型开始出现部分违禁信息、代理信号超过阈值的区域。这种控制视角区别于传统固定prompt路径：它更像是对话中的实时决策，而非事先设定好的多条模板。

不确定性代理信号建模

我们重点设计**黑盒可计算的agent信号**，而不直接依赖logits熵。选取以下主要信号：

1. **Paraphrase Sensitivity（同义改写敏感度）**：给当前提示做轻微改写（保持语义），检查输出是否剧变。实现：对用户最新提示x_t构造几种同义词或语序变体 $\tilde{x}_t^{(j)}$，固定上下文多次生成回复，计算回复类别或语义差异。如从 R0→R2 的变化说明在边界附近。定义度量如：

$$U_{\text{para}}(t) = \frac{1}{M}\sum_{j=1}^{M} \text{dist}(f(h_t, \tilde{x}_t^{(j)}),\ f(h_t, x_t))$$

   其中 dist 可用回复类别变化或embedding距离。此信号突出边界敏感性：若微扰引发模型从拒绝到配合，说明当前状态易受刺激。

2. **Refusal Fragility Score（拒答脆弱度）**：将模型回复按拒绝程度离散分级（如R0=坚拒、R1=拒答+建议、R2=模糊边界、R3=几乎有害、R4=完全有害）。然后用本轮或相邻轮的标签分布来度量不确定性。比如定义：

$$U_{\text{frag}}(t) = H\big(\{C(y_t^{(i)})\}_{i=1}^{K}\big)$$

   或简单取平均级别。这里 $C(y)$ 是回复标签，K 是采样数。脆弱度高意味着回复类别本轮内有跳变，模型处于摇摆。此方法与**semantic entropy**不同，因为它是基于**语义/类别**的离散不确定性，易于理解且与越狱目标直接相关。实现上可以用固定规则或二分类器自动标注回复为 R0–R4，代价低于全量采样，对在线使用友好。

3. **Judge Disagreement（审判分歧）**：使用一个或多个外部判定模型对当前回复从不同角度打分：如安全度评分 J_{safety} 和达标度评分 J_{goal}。若两个评分相差较大，则视为高不确定性：

$$U_{\text{judge}}(t) = |J_{\text{safety}}(y_t) - J_{\text{goal}}(y_t)|.$$

   或多judge方差。此信号捕捉“多重评判下的模糊区间”，例如GPT-4认为已经偏向违规，但Claude给出模棱两可的分数，就说明状态在边缘。PAIR/TAP曾使用单一judge反馈优化提示，我们这里把**Judge分歧本身**作为控制信号。

这些信号应**在线**分层使用：常轮使用轻量的Fragility和Judge分数，在检测到边界倾向时再触发Paraphrase或采样重复检测以获得更稳定的不确定度估计。我们还将考察其他扩展信号如“拒绝词汇波动”、“推理链不一致性”、“自我矛盾检测”等，判断是否增益。这些代理信号各有优劣：Fragility易实现、低开销，Judge分歧能捕获语义层次冲突，Paraphrase敏感度最精准但成本更高。最终可能将多种指标合并为复合得分，用于阶段切换决策。

多 Agent 攻击器设计

我们可以将攻击器拆解为内部多Agent协作结构，以提高攻击规划与生成能力，**但目标系统仍为单一黑盒chatbot**。推荐架构包括：

- **Planner Agent**：监控当前历史、阶段信号与不确定性分数，决定阶段和策略。输入：h_t, z_t, u_t；输出：阶段决策与下一步策略指令（如“继续伪装→”或“进入Bridge阶段→”）。

- **Generator Agent**：根据Planner指示（当前阶段与目标）生成候选提示。比如Destabilize阶段生成类比/铺垫问题，Bridge阶段生成合法化敏感问题，Stabilize阶段生成续写整理的提示。使用攻击者LLM生成多条候选提示集合供评估。

- **Evaluator Agent**：分析目标模型回复，估计拒绝等级、有害程度和不确定性代理（如fragility分数）。输入：回复文本；输出：不确定性得分、提议的阶段切换建议。

- **Memory/Trajectory Agent**（可选）：压缩记录重要的交互信息、有效策略和常见失败模式，为后续决策提供背景支持。

引入多Agent的优势是分工与并行：Generator探索多个提示方向，Evaluator精细评估安全级别，Planner总结决策。相比单一攻击LLM，多Agent能并行试探多种策略（类似Siege的多臂探索 [12] ）并及时汇报。实验中我们会比较单Agent对比多Agent，实现多Agent并行搜索后对比ASR和查询效率差异。但需注意：多Agent设计应为攻击控制器实现细节，而非本文主题核心。主题核心仍在于**不确定性门控的阶段控制**，多Agent更多作为可选增效模块与后续迁移桥梁。

控制策略设计

我们把策略形式化为 $a_t = \pi(h_t, u_t, z_t)$。首先设计可解释的**规则策略**：

- 若 u_t 低且当前回复仍是完全拒绝，则保持Destabilize，生成下一层伪装问题。
- 若 u_t 高且回复出现部分配合（R2/R3），则切换到Bridge，生成桥接型提示。
- 若回复到达near-harmful且连贯性维持，切换到Stabilize。
- 若在Stabilize阶段发现模型回复回退（fragility下降），可重新切换到Bridge保持进度。

优点：简单明了、易调参、可复现；缺点：阈值敏感、可能对新环境泛化差。我们还会探索**学习策略**：将阶段选择视为决策问题，用上下文Bandit或强化学习优化策略。或用MCTS仿真不同阶段动作的可能回报（参考Siege BFS思路 [12] ）。然而，在第一篇工作中，我们倾向于先用规则+局部搜索（如对Bridge阶段多候选提示评分）验证思路，再考虑学习方法。

基线对比

实验中，我们将与以下基线系统对比，论证方法优势：

1. **单轮重试（Repeated single-turn）**：在相同查询预算下，反复尝试最强单轮prompt，检验多轮真的优于单轮多次重试的效果 [10] 。

2. **Crescendo式固定渐进** [1] [5] ：多轮对话逐步升级提示，不用反馈控制，只是强度递增。如Russinovich等的实现。

3. **固定阶段多轮（RACE/FITD）**：RACE [4] 使用固定状态机和预设模板进行多轮攻击，FITD按“先小后大”原则询问。这些方法验证渐进多轮的重要性，但不利用在线信号。

4. **AutoDAN/PAIR/TAP 等黑盒优化** [10] ：与本方法相比，它们在单轮prompt上优化查询但不关注多轮状态，应对比提示级优化与轨迹级控制的差别。

5. **State-machine 迭代方法**（如其它多轮状态机攻击）：例如將Goat、CoA（逐层对话链）等归入这一类。

6. **消融基线**：包括无不确定性控制的固定三阶段、无Stabilize阶段的两阶段攻击、随机阶段切换版本、单一信号 vs 复合信号、单Agent vs 多Agent攻击器等。通过这些对比可以量化每个设计的价值。

对每个基线，需说明核心机制、优点、局限及与本研究差异。例如Crescendo和Goat强调固定策略、多轮升级，但忽略了越狱后稳定化阶段；PAIR/TAP专注单轮高效搜索而缺乏多轮阶段概念等。

实验设计与基准

实验设置

- **目标**：阶段一仅使用普通聊天模型（黑盒API）。
- **数据**：优先使用AdvBench/JailbreakBench的含攻击性指令子集。可控制任务类型、分开类别，或挑选适合多轮渐进的样本子集。
- **目标模型**：至少2-4个不同厂商的模型。例如 OpenAI ChatGPT/GPT-4、Anthropic Claude系列、Meta Llama 等，覆盖开源与闭源、不同对齐策略。考虑模型多样性，以避免方法过拟合某一模型特性。
- **攻击器（Attacker）与Judge**：攻击器LLM可使用ChatGPT/GPT-4或其他强模型；Judge可以选用（或组合）一个高质量模型（如GPT-4o）来评估回复的安全性和目标达成度，警惕单一judge偏差。Judge评估分数用作不确定性信号之一。
- **指标**：主要度量 Attack Success Rate (ASR)（最终输出含有害信息的比例）；Multi-turn ASR（多轮中任意轮成功即可）；部分服从率（Partial ASR，即模型开始泄露有害信息的比例）；Partial→Complete 转化率（在出现部分服从的对话中最终完成度）[2]；平均对话轮数；查询开销。另定义**稳定性分数**（Stability Score），衡量模型首次偏离后能否持续保持：例如在首次出现near-harmful后连续k轮内保持回复质量（使用有害内容的比例或judge评分）不下降。
- **消融实验**：包括去除uncrtainty控制、去除Stabilize阶段、比较不同代理信号、使用单信号对比复合信号、单Agent vs 多Agent等，以验证H1–H5假设：模型是否存在脆弱窗口、不确定性信号与成功率是否显著相关、动态切换是否优于静态逐步推进、Stabilization是否提升完成率、多Agent是否带来额外收益等。

实验结果需统计各基线在各指标上的表现曲线，并分析为何本方法优于基线。例如应展示：在无Stabilize版本中，攻击率可能接近但大部分是“中间成功”而非“完成输出”，而加入Stabilize阶段后完整成功率显著提升[2]；动态切换版本比固定方案更善用少量查询达到更高ASR[10]等。

不确定性代理详解

针对核心代理信号**Paraphrase Sensitivity、Refusal Fragility、Judge Disagreement**，以及可能的组合，我们详细设计并评估它们的数学定义、实现和计算成本等：

- **Refusal Fragility**：定义为回复标签的分布不确定性，如本轮多采样下类别熵 $H(C)$ 或连续轮类别跳变 ΔC。实现成本低（不需额外模型查询，只需分类）且在线性好。我们将设定自动标注规则或使用安全/有害分类器归类回复，并构造 fragility 分数。例如：

  $$u_{\text{frag}}(t) = \alpha\,\overline{C(y_t)} + (1-\alpha)\,H\big(\{C(y_t^{(i)})\}\big).$$

  其关联机制：fragility高的回复意味着模型处于摇摆，预示后续可能更易突破。风险：标签偏差或标注不准确可能误导控制。

- **Paraphrase Sensitivity**：定义对最新提示x_t做若干语义等价改写 $\{\tilde{x}^{(j)}\}$，生成回复计算差异：

$$u_{\text{para}}(t) = \frac{1}{M}\sum_{j=1}^{M} \text{dist}(f(h_t, \tilde{x}^{(j)}),\ f(h_t, x_t)),$$

可用标签变化或相似度度量。实现需要调用攻击模型M次，成本较高。它可准确捕捉边界：若小扰动即大变动，说明当前状态接近决策边界。这常用在核查何时真正触发阶段切换。风险是生成变体的语义偏移可能不完美，且对语言敏感。

- **Judge Disagreement**：若使用两个维度judge J_1, J_2，定义

$$u_{\text{judge}}(t) = |J_1(y_t) - J_2(y_t)|.$$

或多judge方差。比如一个judge评估安全度，另评估任务完成度，不一致说明模糊状态。实现成本是调用judge模型次数，偏差来自judge本身。

我们将对比单一信号与混合信号的控制效果。例如Fragility+Judge或Fragility+Paraphrase的组合，是否比任意单一信号更稳。若最终需要，我们可以构建一个综合不确定度分数：如 $u_t = w_1 u_{\text{frag}} + w_2 u_{\text{para}} + w_3 u_{\text{judge}}$，再由阈值或策略决策切换。这些计算和实验分析将明确：哪些代理最适合作为阶段切换单信号，哪些宜做组合，是否需自适应加权。根据现有分析，[10]强烈提示单信号有限，我们倾向使用复合指标。

实验设计与基准对比

我们将在统一框架下执行对比实验，确保可复现性。具体包括：

- **实验环境**：使用同一数据集（如JailbreakBench子集），同一轮数上限和token预算，对比所有方法。
- **评测指标**：Attack Success Rate、Partial ASR、Partial→Final转化率、平均轮数、查询成本、Stability Score等。Stability Score可定义为：在模型首次进入near-harmful后的若干轮内输出有害内容的持续程度（例如连续k轮内有害字数占比）。
- **基线对比**：如上文所述几类。每个基线保证查询预算对齐，并使用相同judge评估体系。
- **结果分析**：重点检验不确定性控制的价值：比如展示动态 vs 固定阶段在同查询数下的ASR曲线；展示无Stabilize vs 有Stabilize在完成率上的差距；绘制示例对话轨迹，突出当代理信号触发阶段切换时的模型行为变化。

创新点总结

本研究的贡献可以总结为：

- **动态阶段化控制**：将黑盒多轮越狱从单条prompt问题提升为动态控制问题[10]。通过引入阶段变量 z_t 和外部信号 u_t，提出了一个有状态的攻击框架（Destabilize–Bridge–Stabilize），弥补了现有固定化方法的不足。
- **黑盒不确定性代理**：设计并系统评估了一组可在线计算的代理信号（fragility、paraphrase敏感度、judge分歧等）用于阶段切换决策，这是黑盒环境下首次把**模型外部的不确定性**纳入越狱攻击控制。
- **攻击态稳定化**：提出Stabilize阶段的重要性，指出多轮越狱成功不仅要“推入脆弱区”，更要“维持攻击态”。如WASP所示，许多攻击只做到中间偏离[2]；我们着重研究Stabilization策略及其对最终成功率的贡献，这是当前越狱研究中未被关注的环节。
- **可迁移攻击控制器**：设计了一个模块化的攻击控制器架构（含Planner/Generator/Evaluator等），为后续向Web Agent/工作流攻击研究迁移提供基础。未来可无缝将这些模块应用于工具使用型代理，映射Destabilize→lure、Bridge→redirect、Stabilize→maintain等。

多Agent攻击器可提高效率，但其本身不构成主要贡献。我们重点放在**不确定性门控的多轮控制**上，多Agent只作为可选优化。最终形成的控制框架和信号机制将填补现有越狱研究的空白。

可行性、风险与扩展

可行性：初期开发难度在于实现统一对话框架和信号计算，但大部分工作可在数周内实现MVP。当前公开环境（如ChatGPT API或本地LLaMA）支持多轮对话测试。**工程难点**是多轮历史管理、并行Probe实现和查询预算控制。我们将优先实现基于阈值的控制策略，不依赖复杂RL，以便结果可靠。

开销风险：引入代理信号必然增加查询成本。Mitigation：采用事件触发式Probe（仅在怀疑窗口时多采样），并使用快速Judge。我们会限制每轮额外采样次数（如 2–3 次）并综合计算总体开销。将密切监测**效费比**，确保大幅提升ASR的同时查询成本合理。

负面风险：若发现精心设计的代理信号实际上不优于简单方法，我们不应强求“entropy是关键”。备选方案是将研究重点调为“评估一系列黑盒不确定性信号作为多轮攻击控制依据”的探索性视角，而非一成不变地宣传熵指标本质。即使单一信号作用有限，我们也可强调对比分析与信号组合的价值。

扩展讨论：第一阶段完成后，我们将在聊天机器人上稳定下控制策略，再向Web Agent迁移。大部分组件（Planner, 信号逻辑, 切换规则）可直接复用，只是观测内容从纯文本扩展到工具输出、行动序列等。相比直接早期投入复杂代理场景，这一步后续可减少开发风险和变量干扰。我们会分析Destabilize→lure、Bridge→redirect/commit、Stabilize→maintain的对应关系，链接WASP、AgentDojo等Agent安全评测工作[2][13]。

综上，本研究在**动态控制**、多轮**不确定性代理**、**Stabilization** 等方面开创新思路，并为后续向多agent工作流越狱提供可行基础。所有设计都将以严谨、可复现的实验为支撑，遵循ACM/IEEE风格顶会可接受的标准。

---

[1] usenix.org
https://www.usenix.org/system/files/conference/usenixsecurity25/sec25cycle1-prepub-805-russinovich.pdf

[2] arxiv.org
https://arxiv.org/pdf/2504.18575

[3] Logo_v3 Foot-In-The-Door: A Multi-turn Jailbreak for LLMs
https://fitd-foot-in-the-door.github.io/

[4] [6] Knowledge-Driven Multi-Turn Jailbreaking on Large Language Models
https://arxiv.org/pdf/2601.05445

[5] [12] Siege: Autonomous Multi-Turn Jailbreaking of Large Language Models with Tree Search
https://arxiv.org/html/2503.10619v1

[7] [2506.16792] MIST: Jailbreaking Black-box Large Language Models via Iterative Semantic Tuning
https://arxiv.org/abs/2506.16792

[8] [2310.04451] AutoDAN: Generating Stealthy Jailbreak Prompts on Aligned Large Language Models
https://arxiv.org/abs/2310.04451

[9] State-Dependent Safety Failures in Multi-Turn Language Model Interaction
https://arxiv.org/html/2603.15684v1

[10] [2508.07646] Multi-Turn Jailbreaks Are Simpler Than They Seem
https://arxiv.org/abs/2508.07646

11 NeurIPS Poster Tree of Attacks: Jailbreaking Black-Box LLMs Automatically
https://neurips.cc/virtual/2024/poster/95078

13 ethz-spylab/agentdojo: A Dynamic Environment to Evaluate Attacks ...
https://github.com/ethz-spylab/agentdojo